

VAPT sull'immagine Docker di OWASP Juice Shop

In questa relazione viene descritto il processo di Vulnerability Assessment e Penetration Test (VAPT) condotto sull'applicazione web OWASP Juice Shop, eseguita tramite container Docker.

OWASP Juice Shop è un'applicazione web che presenta volontariamente al suo interno numerose vulnerabilità per permettere agli sviluppatori di esercitarsi.

L'obiettivo del progetto è stato identificare, analizzare e documentare le vulnerabilità dell'immagine Docker, seguendo le best practice del penetration testing.

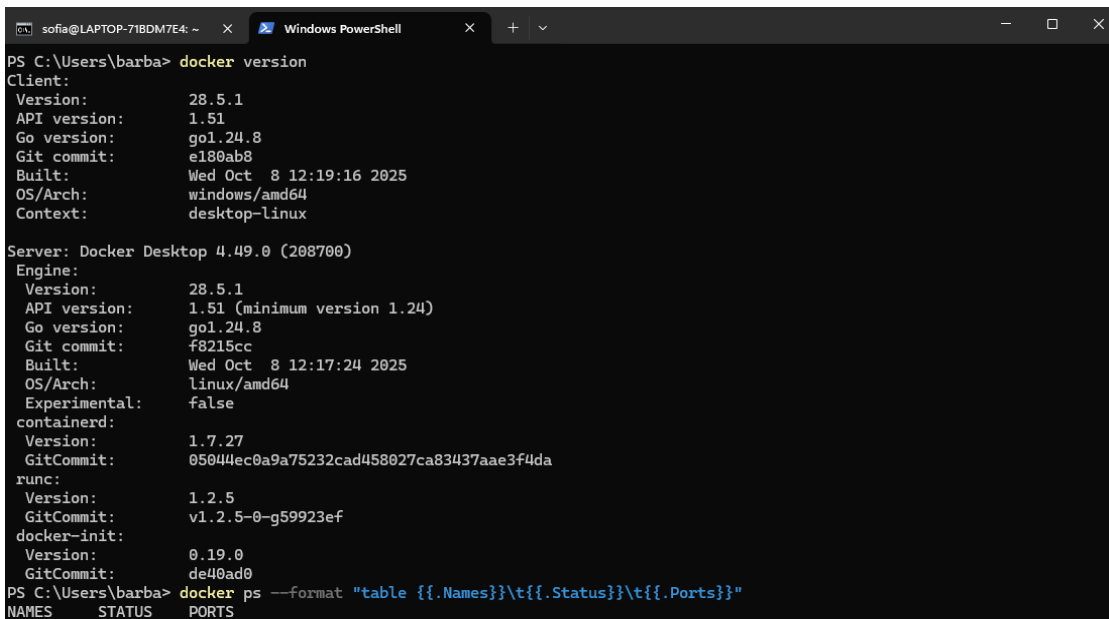
La metodologia adottata segue le principali fasi del penetration testing:

- **Information Gathering:** Raccolta delle informazioni sull'applicazione e il suo ambiente.
- **Vulnerability Assessment:** Identificazione delle potenziali vulnerabilità.
- **Exploitation:** Tentativi di sfruttamento delle vulnerabilità individuate.
- **Post-Exploitation:** Analisi degli impatti e delle possibilità di persistenza o escalation.
- **Report:** raccolta e spiegazione dei risultati ottenuti

Durante il VAPT sull'applicazione OWASP Juice Shop, l'ambiente di test è stato configurato tramite Docker Desktop su Windows. Le attività di analisi sono state condotte mediante strumenti automatici (OWASP ZAP, Nikto, nmap) e manuali (Browser Chrome + DevTools), supportati da comandi Docker e PowerShell per il monitoraggio dei log e la raccolta delle evidenze. Tutte le prove sono state documentate e raccolte nel pacchetto ZIP finale contenente output, HAR, screenshot e file di log.

Comandi di inizializzazione

Come primo passo, dopo aver installato Docker Desktop ed averne verificato l'esistenza con i comandi, `docker version` e `docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"` ;



```
PS C:\Users\barba> docker version
Client:
 Version:      28.5.1
 API version:  1.51
 Go version:   go1.24.8
 Git commit:   e180ab8
 Built:        Wed Oct  8 12:19:16 2025
 OS/Arch:      windows/amd64
 Context:      desktop-linux

Server: Docker Desktop 4.49.0 (208700)
Engine:
 Version:      28.5.1
 API version:  1.51 (minimum version 1.24)
 Go version:   go1.24.8
 Git commit:   f8215cc
 Built:        Wed Oct  8 12:17:24 2025
 OS/Arch:      linux/amd64
 Experimental: false
 containerd:
 Version:      1.7.27
 GitCommit:    05044ec0a9a75232cad458027ca83437aae3f4da
 runc:
 Version:      1.2.5
 GitCommit:    v1.2.5-0-g59923ef
 docker-init:
 Version:      0.19.0
 GitCommit:    de40ad0
PS C:\Users\barba> docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

scarico e avvio l'immagine Juice Shop e ne verifico l'esecuzione attraverso i seguenti comandi:

- `docker pull bkimminich/juice-shop:latest`
- `docker run -d --name juice-shop -p 3000:3000 bkimminich/juice-shop:latest`

Verifico che il container sia in esecuzione con il comando

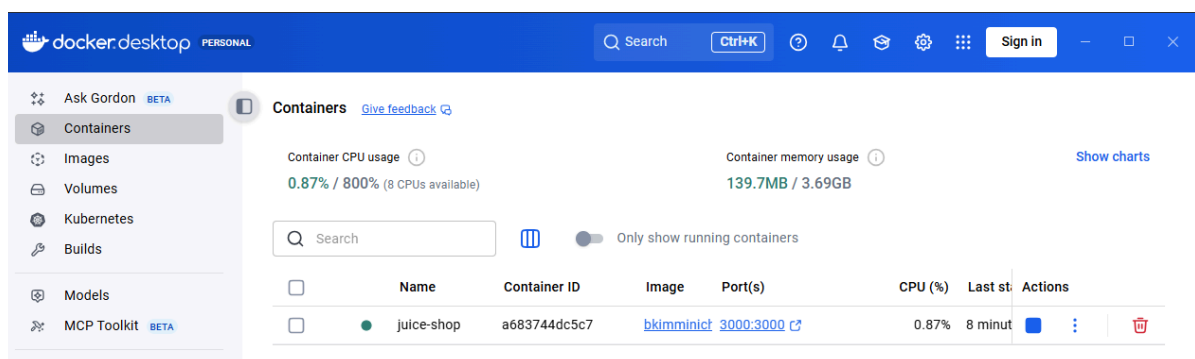
- `docker ps --format "table {{.Names}}\t{{.ID}}\t{{.Ports}}"`

```
PS C:\Users\barba> docker pull bkimminich/juice-shop:latest
latest: Pulling from bkimminich/juice-shop
4aa0ea1413d3: Pull complete
c058825cfd6: Pull complete
33ce0b1d99fc: Pull complete
4eff9a62d888: Pull complete
3214acf345c0: Pull complete
045fc1c20da8: Pull complete
46b46b3ee13c: Pull complete
5b14f6c9a813: Pull complete
ddf74a63f7d8: Pull complete
c223915a7ea9: Pull complete
bfb59b82a9b6: Pull complete
35d697fe2738: Pull complete
da7816fa955e: Pull complete
f45e0372ce60: Pull complete
2780920e5dbf: Pull complete
5664b15f108b: Pull complete
7faf0cfa885c: Pull complete
62de241dac5f: Pull complete
d00c3209d929: Pull complete
7c12895b777b: Pull complete
e27f89e01: Pull complete
Digest: sha256:c6f965f8929c2c43676e3ac55cd19d482c0084400195db07ed7513a04f3468b5
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest
PS C:\Users\barba> docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Size}}"
REPOSITORY          TAG          IMAGE ID      SIZE
bkimminich/juice-shop latest       c6f965f8929c 661MB
PS C:\Users\barba> docker run -d --name juice-shop -p 3000:3000 bkimminich/juice-shop:latest
a683744dc5c7e7332067b4334c69a2bc877ac1ef76a7020a18d305245991e981
PS C:\Users\barba> docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES              STATUS      PORTS
juice-shop         Up 14 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
```

Verifico che il Docker funzioni anche da Ubuntu.

```
sofia@LAPTOP-71BDM7E4:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
a683744dc5c7   bkimminich/juice-shop:latest        "/nodejs/bin/node /j..." 18 minutes ago Up 10 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
sofia@LAPTOP-71BDM7E4:~$
```

A questo punto, possiamo aprire l'applicazione web che si trova attiva sulla porta localhost:3000 .



Preparazione al lavoro

Prima di iniziare, ho creato una cartella (juice_shop_evidence_package), nella quale raccogliere tutti gli output e le loro immagini relative.

Installo, su Ubuntu, gli strumenti per effettuare le fasi di Information Gathering.

Quindi, con i comandi, `sudo apt update` e `sudo apt install -y nmap nikto whatweb gobuster curl`, installo:

- **nmap** che mi servirà per la scansione di porte e servizi
- **nikto** per scannerizzare le vulnerabilità web
- **whatweb** il quale identifica framework e tecnologie
- **gobuster** per trovare directory e file nascosti su un sito web, ma anche sottodomini
- **curl** per le richieste HTTP manuali

Tramite il comando `docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' juice-shop`, individuo il target IP.

```
sofia@LAPTOP-71BDM7E4:~$ docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' juice-shop
172.17.0.2
```

Information Gathering

Durante la fase di **Information Gathering** sono stati avviati controlli di raggiungibilità e scansioni con strumenti tipici (nmap, whatweb, nikto), è stata eseguita l'enumerazione di directory (con uno script curl in luogo di gobuster), è stata raccolta la lista degli header HTTP esposti e sono state salvate tutte le evidenze in una struttura di cartelle.

Alcuni test manuali preliminari (usando l'IP interno del container `172.17.0.2`) fallivano per problemi di raggiungibilità; usando invece `localhost:3000` (porta mappata) le scansioni sono andate a buon fine e hanno evidenziato più risorse interessanti (`/ftp`, `/score-board`, `/api`, `/rest`, `robots.txt`, ecc.).

Prima di procedere con le varie operazioni, ho creato una cartella attraverso il comando `mkdir -p ~/juice_shop_vapt/evidence/{nmap,whatweb,nikto,gobuster}`, per racchiudere tutte le informazioni trovate.

Nmap (le informazioni raccolte si trovano nel percorso `...\evidence\nmap`)

I comandi **nmap** sono stati usati per identificare porte aperte e servizi in ascolto sull'host. Sono stati effettuati due tentativi di scansione:

- `nmap -p 3000 -sV 172.17.0.2`
 - `-p 3000`: scansiona la porta TCP 3000, nota per l'applicazione Juice Shop.
 - `-sV`: rileva la versione del servizio che risponde sulla porta.
 - L'output è stato salvato in `evidence/nmap/nmap_scan_basic.txt`.

- `sudo nmap -A 172.17.0.2 -p 3000`
 - `-A`: modalità "aggressiva" che combina rilevamento versione, sistema operativo, script NSE e traceroute. Fornisce informazioni più dettagliate ma richiede che l'host sia raggiungibile.

```
sofia@LAPTOP-71BDM7E4:~/juice_shop_vapt$ nmap -p 3000 -sV 172.17.0.2 -oN evidence/nmap/nmap_scan_basic.txt
Starting Nmap 7.80 ( https://nmap.org ) at 2025-10-24 12:30 CEST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.42 seconds
sofia@LAPTOP-71BDM7E4:~/juice_shop_vapt$ sudo nmap -A 172.17.0.2 -p 3000 -oN evidence/nmap/nmap_scan_aggressive.txt
[sudo] password for sofia:
Starting Nmap 7.80 ( https://nmap.org ) at 2025-10-24 12:31 CEST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.81 seconds
```

Entrambi i comandi hanno concluso con il messaggio "0 hosts up", non perché Nmap sia difettoso, ma perché non è riuscito a verificare la presenza e la raggiungibilità dell'host inviando i messaggi di test di rete predefiniti.

Le cause più comuni di questo risultato sono:

- `172.17.0.2` è l'IP interno di un container Docker in una rete bridge, spesso non raggiungibile direttamente da WSL o dall'host per isolamento o politiche di rete.
- L'host potrebbe bloccare i probe di rilevamento (firewall, iptables, configurazioni Docker).
- Il container Docker potrebbe non essere attivo.

Il test con `curl -I http://localhost:3000` ha restituito un codice `200 OK`, confermando che il Docker è attivo, la porta 3000 del container è mappata sulla porta 3000 dell'host (`-p 3000:3000`) e la comunicazione host-container tramite `localhost` funziona correttamente. La mancata raggiungibilità dell'IP container non significa che il servizio sia inattivo, ma che l'host non è raggiungibile tramite l'indirizzo IP del container. Per questo, nel penetration test è importante basare l'analisi su `localhost:3000`.

Quando Nmap rileva la porta aperta e il servizio attivo (da `localhost`), si possono poi scegliere test mirati di vulnerabilità come SQL injection, XSS o IDOR.

Docker Logs

Dai log del container (ottenuti con `docker logs juice-shop --tail 100`) sono emersi ripetuti errori di runtime, ad esempio: `Error: Unexpected path: /rest at /juice-shop/build/routes/angular.js:42:18`. Questi errori indicano che l'applicazione Express del Juice Shop sta ricevendo richieste su endpoint `/rest` e `/api` che non sono gestiti correttamente, generando errori interni (HTTP 500) e stack trace.

```
sofia@LAPTOP-71BDM7E4:~$ docker logs juice-shop --tail 100 > docker_logs.txt
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at logger (/juice-shop/node_modules/morgan/index.js:144:5)
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
Error: Unexpected path: /rest
at /juice-shop/build/routes/angular.js:42:18
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/routes/verify.js:187:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/routes/verify.js:111:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at logger (/juice-shop/node_modules/morgan/index.js:144:5)
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
```

Implicazioni per il VAPT

Gli errori 500 indicano una gestione degli errori non robusta, potenziali leak di informazioni e possibili comportamenti anomali da sfruttare. Gli endpoint `/rest` e `/api` devono essere analizzati con richieste mirate (con specifici body o header) per verificare la presenza di vulnerabilità come injection o IDOR.

Nikto

Nikto è stato utile per effettuare un'analisi approfondita del server web per identificare varie potenziali vulnerabilità e configurazioni non ottimali, fornendo informazioni dettagliate e utili alla sicurezza dell'applicazione.

Risultati e spiegazione dei passaggi

Nikto ha rilevato header HTTP non standard, come `X-Recruiting: /#/jobs`, che indicano percorsi interni o funzionalità potenzialmente nascoste o poco documentate. Questo header è stato analizzato per comprendere se faccia parte di una funzionalità sensibile o solo informativa.

```
sofia@LAPTOP-71BDM7E4:~/juice_shop_vapt$ nikto -h http://localhost:3000 -output ~/juice_sh
op_vapt/evidence/nikto/nikto_output.txt
- Nikto v2.1.5

-----
+ Target IP:      127.0.0.1
+ Target Hostname: localhost
+ Target Port:    3000
+ Start Time:     2025-10-24 12:42:39 (GMT2)
-----

+ Server: No banner retrieved
+ Server leaks inodes via ETags, header found with file /, fields: 0xW/124fa 0x19a15a334b4
+ Uncommon header 'feature-policy' found, with contents: payment 'self'
+ Uncommon header 'x-frame-options' found, with contents: SAMEORIGIN
+ Uncommon header 'x-content-type-options' found, with contents: nosniff
+ Uncommon header 'x-recruiting' found, with contents: /#/jobs
+ Uncommon header 'access-control-allow-origin' found, with contents: *
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ File/dir '/ftp/' in robots.txt returned a non-forbidden or redirect HTTP code (200)
+ "robots.txt" contains 1 entry which should be manually viewed.
+ Uncommon header 'access-control-allow-methods' found, with contents: GET,HEAD,PUT,PATCH,
POST,DELETE
+ OSVDB-3092: /css: This might be interesting...
+ OSVDB-3092: /ftp/: This might be interesting...
+ OSVDB-3092: /public/: This might be interesting...
+ 6544 items checked: 2 error(s) and 12 item(s) reported on remote host
+ End Time:       2025-10-24 12:45:19 (GMT2) (160 seconds)
-----

+ 1 host(s) tested
```

È stata inoltre identificata la presenza di una directory `/ftp/` segnalata nel file `robots.txt`, alla quale è però possibile accedere pubblicamente con codice HTTP 200. Indicatori come questo spesso segnalano risorse o dati che, anche se non dovrebbero essere pubblici, risultano comunque accessibili e rappresentano un rischio di esposizione.

Un'attenzione particolare è stata data agli header **ETag** emessi dal server, che includono valori legati agli inode del filesystem:

- Questi valori rivelano informazioni sull'architettura interna del server, come il sistema di file e l'ordine di creazione dei file;
- pur non essendo direttamente sfruttabili per un accesso non autorizzato, possono facilitare attacchi di information disclosure;
- è consigliata la disabilitazione o modifica nella configurazione del server per evitare che venga esposta questa informazione, per esempio disabilitando ETag o usando ETag "deboli" privi di riferimenti a inode.

Nikto ha anche individuato header di sicurezza generalmente positivi, ma migliorabili, quali:

- **Feature-Policy**, che limita l'accesso ad API specifiche, come la funzione di pagamento, riducendo l'attacco a funzionalità sensibili;
- **X-Frame-Options**, che previene attacchi di tipo clickjacking consentendo l'inclusione in iframe solo dallo stesso dominio;
- **X-Content-Type-Options**, che impedisce al browser di fare inferenze sul tipo MIME del contenuto, mitigando attacchi XSS basati sul MIME sniffing.

È stato rilevato un header custom **X-Recruiting** che fornisce informazioni su una sezione **/jobs** dell'applicazione. Questo è principalmente informativo, ma può essere utile per la ricognizione.

Un punto critico è stato evidenziato relativamente al meccanismo CORS:

- L'intestazione ****Access-Control-Allow-Origin: *** consente richieste da qualsiasi origine, esponendo l'applicazione a potenziali rischi come furto di dati o azioni non autorizzate via cross-origin.
- sarebbe meglio quindi limitare tale header a domini affidabili tramite configurazioni specifiche, migliorando la sicurezza degli endpoint.

Infine, è stato riscontrato che il server accetta vari metodi HTTP mutativi come PUT, PATCH e DELETE. Questi, se non autenticati e autorizzati, possono rappresentare rischio elevato di **Broken Access Control**, cioè possibilità di accessi e modifiche non lecite.

WhatWeb

WhatWeb è uno strumento open source utilizzato principalmente per identificare le tecnologie web impiegate da un sito, come sistemi di gestione dei contenuti (CMS), piattaforme blog, librerie JavaScript, server web e altri componenti tecnologici. Viene utilizzato per individuare potenziali vulnerabilità o configurazioni errate che potrebbero compromettere la sicurezza dell'applicazione.

Nel caso analizzato, WhatWeb ha rilevato che l'applicazione in esecuzione su **http://localhost:3000** è **OWASP Juice Shop**. Sono state identificate tecnologie come **HTML5**, che rappresentano una pagina web moderna e **jQuery 2.2.4**, una libreria JavaScript ormai datata (serie 2.x). **Librerie obsolete come questa possono contenere bug o funzioni sfruttabili in attacchi di tipo XSS o DOM-based se il front-end è vulnerabile.**

L'analisi ha inoltre evidenziato altri elementi importanti:

- **Script module:** indica l'uso di script con `type="module"` o moduli JS ES6, che cambiano il modo in cui gli script vengono caricati, influenzando così le tecniche di sfruttamento DOM.
- **Uncommon Headers:** segnala header HTTP non standard o rilevanti presenti nella risposta.
- **X-Frame-Options: SAMEORIGIN e Feature-Policy: payment 'self'** sono header di sicurezza che regolano il framing della pagina e l'uso di API privilegiate nel browser, contribuendo alla protezione contro attacchi come il clickjacking (tipo di attacco dove la vittima fa un click su un sito che, all'apparenza sembra affidabile, ma in realtà è malevolo).

Rischi potenziali

- **jQuery obsoleta:** la versione 2.2.4 può rappresentare una **superficie di attacco per vulnerabilità note**, specialmente relative a manipolazioni del DOM (es. uso non sicuro di `.html()` con dati non sanitificati).
- ****Access-Control-Allow-Origin: ***:** questo header consente richieste XHR da qualsiasi origine, aumentando il rischio in scenari cross-origin. Ciò **può facilitare attacchi di tipo data exfiltration** (furto di dati) o **CSRF-like** (attacchi da parte di un aggressore che portano la vittima ad effettuare delle richieste contro la sua volontà) in combinazione con token deboli o cookie non protetti.

Analisi dettagliata degli header HTTP rilevati

```
HTTP Headers:
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Fri, 24 Oct 2025 09:53:28 GMT
ETag: W/"124fa-19a15a334b4"
Content-Type: text/html; charset=UTF-8
Vary: Accept-Encoding
Content-Encoding: gzip
Date: Fri, 24 Oct 2025 10:41:57 GMT
Connection: close
Transfer-Encoding: chunked
```

Header

Significato e Implementazioni

Header	Significato e Implementazioni
Access-Control-Allow-Origin: *	Permette richieste cross-origin da qualsiasi dominio. Rischia di esporre dati se non correttamente controllato.

X-Content-Type-Options: nosniff	L'header nosniff impedisce al browser di interpretare male i contenuti di determinati file, costringendolo a rispettare il tipo MIME dichiarato dal server e quindi a trattare il contenuto esattamente come previsto
X-Frame-Options: SAMEORIGIN	Impedisce il framing da domini esterni, proteggendo dal clickjacking.
Feature-Policy: payment 'self'	Limita l'uso della Web Payments API al dominio dell'applicazione. Questo header specifica che solo la pagina web principale, e i suoi iframe nello stesso dominio, possono accedere e utilizzare l'API per i pagamenti digitali, impedendo invece il suo uso da origini esterne o di terze parti.
X-Recruiting: /#/jobs	Header custom che segnala una sezione "jobs" nell'app. Non è una vulnerabilità ma potrebbe rivelare informazioni sull'applicazione le quali potrebbero essere usate da un attaccante o un tester, per capire se esistono endpoint di interesse da analizzare o eventualmente provare a esplorare per eventuali vulnerabilità o dati sensibili.
ETag: W/"124fa-19a15a334b4"	Usato per il caching (archiviazione di dati e file). Potrebbe portare ad un potenziale leak di informazioni interne se l'ETag non è generato correttamente.
Content-Encoding: gzip / Transfer-Encoding: chunked	Utili per interpretare correttamente il corpo delle risposte HTTP e inviarle in blocchi di dimensioni variabili senza indicare in anticipo la lunghezza totale del contenuto.

Suggerimenti di sicurezza

- Verificare attentamente l'uso di jQuery 2.2.4, cercando riferimenti nel codice sorgente (ad esempio nel pannello Sources o Elements di DevTools) e salvando il file per analisi approfondite.
- Ridurre la permissività del CORS ([Access-Control-Allow-Origin](#)) limitando le origini consentite o implementando controlli server-side sull'header Origin.
- Mantenere attivi header di protezione come [X-Content-Type-Options](#) e [X-Frame-Options](#), valutando la possibilità di introdurre anche una Content-Security-Policy (CSP) per mitigare ulteriormente XSS e caricamenti script non autorizzati.
- Monitorare e valutare header personalizzati per evitare di esporre informazioni sensibili sulla struttura dell'applicazione.
- Controllare il modo in cui sono generati gli ETag per evitare leak di informazioni che potrebbero facilitare il fingerprinting del sistema.

Curl

Il comando `curl`, serve per analizzare gli header HTTP restituiti dal server dell'applicazione, eseguendo la richiesta `curl -I http://localhost:3000` (risultati nel file: `ig_curl_localhost.txt`). L'obiettivo era verificare la raggiungibilità del servizio, osservare le intestazioni di risposta e raccogliere informazioni utili sulla configurazione del server e sulle tecnologie impiegate.

L'output ha mostrato che il server risponde correttamente con codice HTTP/1.1 200 OK, confermando la disponibilità dell'applicazione, e include una serie di header di sicurezza e configurazione, tra cui `Access-Control-Allow-Origin: *`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Feature-Policy: payment 'self'` e `X-Recruiting: /#/jobs`.

```
sofia@LAPTOP-71BDM7E4:~$ curl -I --max-time 5 http://localhost:3000 | tee evidence/ig_curl_localhost.txt
tee: evidence/ig_curl_localhost.txt: No such file or directory
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0 75002    0     0     0     0      0      0  --:--:-- --:--:-- --:--:--     0
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Fri, 24 Oct 2025 09:53:28 GMT
ETag: W/"124fa-19a15a334b4"
Content-Type: text/html; charset=UTF-8
Content-Length: 75002
Vary: Accept-Encoding
Date: Sun, 26 Oct 2025 15:14:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Questi header forniscono indicazioni importanti: ad esempio, l'opzione `Access-Control-Allow-Origin: *` evidenzia una configurazione CORS aperta che, se non gestita correttamente, potrebbe consentire richieste da domini non autorizzati. `X-Frame-Options: SAMEORIGIN` indica invece una protezione contro attacchi di clickjacking; mentre `X-Content-Type-Options: nosniff` riduce il rischio di MIME sniffing.

L'analisi con `curl` ha permesso di ottenere informazioni tecniche sul comportamento del server e sulle sue impostazioni di sicurezza, senza necessità di strumenti complessi.

Gobuster e WordLists

Gobuster è un tool che esegue il brute-forcing di URL, percorsi, vhost, nomi, file su un web server. La wordlist è il file contenente le parole che gobuster prova una per una contro l'URL target. Senza wordlist gobuster non avrebbe nulla da inviare.

Il file **mini_dirs.txt** è una wordlist personalizzata creata per simulare l'utilizzo di strumenti di directory brute forcing durante la fase di Information Gathering. In condizioni normali, tali strumenti utilizzano liste molto ampie di nomi di directory e file comuni per individuare risorse accessibili o nascoste all'interno di un'applicazione web.

```
sofia@LAPTOP-71BDM7E4:~/juice_shop_vapt$ cat > ~/juice_shop_vapt/evidence/wordlists
/mini_dirs.txt <<'EOF'
admin
ftp
rest
api
score-board
assets
uploads
robots.txt
sitemap.xml
login
logout
users
orders
basket
static
images
public
docs
health
EOF
```

Poiché in questo contesto non era possibile installare wordlist esterne o pacchetti aggiuntivi sul sistema, ho costruito manualmente una versione ridotta contenente i percorsi più rilevanti e potenzialmente sensibili per l'applicazione target OWASP Juice Shop. La wordlist **mini_dirs.txt** include voci tipiche come *admin*, *ftp*, *api*, *rest*, *uploads*, *robots.txt*, *users*, *orders*, *login* e altre directory comunemente utilizzate per funzioni amministrative o di backend. Ogni voce rappresenta un possibile percorso accessibile sul server web, che viene successivamente testato tramite richieste HTTP automatizzate (in questo caso, con **curl**).

L'obiettivo del file è quindi quello di **guidare l'enumerazione delle directory** in modo mirato, consentendo di identificare:

- pagine o sezioni sensibili non direttamente collegate nell'interfaccia principale;
- endpoint di API e moduli di amministrazione potenzialmente vulnerabili;
- risorse pubbliche o non protette (es. **/ftp/**, **/uploads/**, **/public/**).

```
sofia@LAPTOP-71BDM7E4:~/juice_shop_vapt$ cd ~
TARGET="http://localhost:3000" # o http://172.17.0.2:3000
IN=~/juice_shop_vapt/wordlists/mini_dirs.txt
OUT=~/juice_shop_vapt/evidence/gobuster/gobuster_output_manual.txt

> "$OUT" # svuota il file
while IFS= read -r path; do
  url="${TARGET%}/$path"
  # esegui HEAD, prendi http_code e Location se c'è redirect
  resp=$(curl -s -I --max-time 8 "$url")
  code=$(echo "$resp" | awk '/^HTTP/{print $2; exit}')
  location=$(echo "$resp" | awk -F: ' /Location:/ {print $2; exit}')
  printf "%s %s %s\n" "$code" "$url" "${location:-}" >> "$OUT"
done < "$IN"

# mostra gli ultimi risultati
tail -n 200 "$OUT"
200 http://localhost:3000/admin -
200 http://localhost:3000/ftp -
500 http://localhost:3000/rest -
500 http://localhost:3000/api -
200 http://localhost:3000/score-board -
301 http://localhost:3000/assets /assets/
200 http://localhost:3000/uploads -
200 http://localhost:3000/robots.txt -
200 http://localhost:3000/sitemap.xml -
200 http://localhost:3000/login -
200 http://localhost:3000/logout -
200 http://localhost:3000/users -
200 http://localhost:3000/orders -
200 http://localhost:3000/basket -
200 http://localhost:3000/static -
200 http://localhost:3000/images -
200 http://localhost:3000/public -
200 http://localhost:3000/docs -
200 http://localhost:3000/health -
```

Con il comando `cd ~TARGET="http://localhost:3000"`
`IN=~/.juice_shop_vapt/wordlists/mini_dirs.txt`
`OUT=~/.juice_shop_vapt/evidence/gobuster/gobuster_output_manual.txt` > "\$OUT"
si specificano le variabili su cui lavorare e si crea o svuota il file di output
`gobuster_output_manual.txt` in modo da partire da zero

Il ciclo `while IFS= read -r path; do ... done < "$IN"`: legge la wordlist riga per riga, assegnando ogni riga ad una `path`. Quindi successivamente si genera un url unendo `TARGET` e `path`.

200 → risorsa **esistente** e accessibile

301 + Location → redirect: segui la Location e verifica la destinazione.

500 → errore server: indica un endpoint che non gestisce correttamente degli input.

Creo la cartella `evidence/gobuster_content` dove salvare i contenuti.

Per ogni `path` uso `curl -s -o /dev/null -w "%{http_code}"` per ottenere solo il codice HTTP. Se il `code` è **200**, riscrive la riga in `gobuster_output_manual.txt` e scarica le prime 200 righe del body della pagina (`curl ... | head -n 200`) in un file con nome derivato dal `path`. Se non è 200, scrive comunque `code url` nel file di output.

Ottenere il body (o almeno un'anteprima) per le pagine che esistono è utile per vedere se `/admin` è una pagina di login o una dashboard aperta, leggere `robots.txt` e `sitemap.xml` e valutare se `/ftp/` contiene file.

```
sofia@LAPTOP-71BDM7E4:~$ TARGET="http://localhost:3000"
OUT_DIR=~/.juice_shop_vapt/evidence/gobuster_content
mkdir -p "$OUT_DIR"

while IFS= read -r path; do
    url="${TARGET%}/$path"
    code=$(curl -s -o /dev/null -w "%{http_code}" --max-time 8 "$url")
    if [ "$code" = "200" ]; then
        echo "200 $url" >> ~/.juice_shop_vapt/evidence/gobuster/gobuster_output_manual.txt
        curl -s --max-time 8 "$url" | head -n 200 > "$OUT_DIR/$(echo $path | sed 's/[a-zA-Z0-9_-]/_/g')
.txt"
    else
        echo "$code $url" >> ~/.juice_shop_vapt/evidence/gobuster/gobuster_output_manual.txt
    fi
done < ~/.juice_shop_vapt/wordlists/mini_dirs.txt
```

Questo processo serve a scoprire risorse accessibili, percorsi nascosti e possibili punti di ingresso o vulnerabilità dell'applicazione target.

Perché si analizzano queste risorse e a cosa servono

1. Mappatura della superficie di attacco

Enumerare le directory e pagine pubbliche o protette (come `/admin`, `/api`, `/ftp`, `/uploads`) consente di identificare l'area complessiva che l'applicazione espone a utenti o potenziali attaccanti. Ogni endpoint o cartella trovata rappresenta un possibile vettore di attacco se non adeguatamente protetto.

2. Individuazione di risorse sensibili o funzionalità critiche

Directory come `/admin` o `/ftp` spesso ospitano funzionalità riservate o file contenenti dati sensibili. Scoprire che questi endpoint rispondono con un codice 200 indica che sono raggiungibili e quindi soggetti a una possibile estrazione di informazioni.

3. Identificazione di risposte anomale

Codici 500 (errore interno del server) per `/rest` e `/api`, segnalano problemi di gestione delle richieste su questi endpoint. Questi errori, se confermati, possono essere sfruttati per rivelare vulnerabilità quali injection, problemi di configurazione o crash.

4. Verifica della presenza di file di controllo e mappe del sito(`robots.txt`, `sitemap.xml`)

Questi file vengono utilizzati dai motori di ricerca e possono nascondere indicazioni su quali risorse il proprietario del sito preferisce non vengano indicizzate, rivelando così aree potenzialmente sensibili.

5. Controlli di autenticazione e sessione (`login`, `logout`, `users`, `orders`, `basket`)

Endpoint che gestiscono utenti e ordini sono critici dal punto di vista della sicurezza e la loro presenza e accessibilità è fondamentale per comprendere il flusso dell'applicazione e identificare possibili punti di attacco, come privilege escalation o business logic flaws.

6. Risorse statiche (`assets`, `static`, `images`) e documentazione (`docs`)

Possono contenere risorse che potrebbero includere commenti, script o configurazioni esposte, utili per svelare ulteriori informazioni sul sistema.

7. Endpoint di salute e monitoraggio (`health`)

Indicano che il sistema include meccanismi di monitoraggio, potenzialmente utili per verificarne lo stato o per identificare qualche informazione durante attacchi di tipo denial of service o availability.

Analisi dei risultati ottenuti

Endpoint	Codice HTTP	Significato
----------	-------------	-------------

<code>/admin</code>	200	Area di amministrazione accessibile; potenziale rischio se non protetta adeguatamente
<code>/ftp</code>	200	Accesso a una directory potenzialmente sensibile o dati caricati
<code>/rest</code>	500	Errore server, vulnerabilità o configurazione errata da investigare
<code>/api</code>	500	Errore server, potrebbe indicare problemi in API backend o possibilità di injection
<code>/score-board</code>	200	Funzionalità o dati visibili, possibile bucket per info di gioco/statistiche
<code>/assets</code>	301 (redirect)	Risorse statiche ridirezionate correttamente
<code>/uploads</code>	200	Cartella di upload utenti, potenziale rischio se non validata
<code>/robots.txt</code>	200	File di controllo che può indicare risorse da non scansionare
<code>/sitemap.xml</code>	200	Mappa del sito, aiuta a scoprire tutte le URL note agli amministratori
<code>/login</code>	200	Pagina login accessibile, area di interesse per test di autenticazione

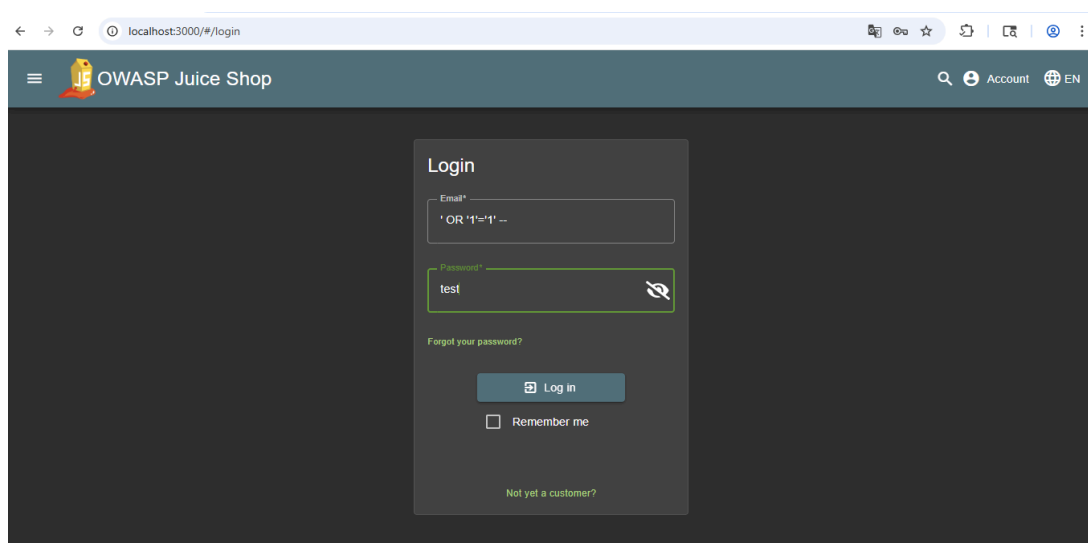
/logout	200	Endpoint di uscita, indica gestione sessioni attiva
/user	200	Endpoint utenti, sensibile per la sicurezza dati e accesso
/orders	200	Endpoint ordini, importante per sicurezza applicativa e business logic
/basket	200	Carrello acquisti, processi critici di business logic
/static, /images	200	Risorse statiche visibili, potenziali vettori di info leak
/public	200	Risorse pubbliche generali
/docs	200	Documentazione, potrebbe includere dettagli di configurazione o AP
/health	200	Stato salute sistema, utile per monitoring e attacchi DoS mirati

Vulnerability Assessment e Penetration Test (VAPT)

Prima di iniziare la fase di test per le vulnerabilità specifiche, è importante comprendere che alcune delle più critiche minacce alle applicazioni web riguardano i meccanismi con cui i dati vengono gestiti e protetti. SQL Injection e Broken Access Control, sono tra le vulnerabilità più comuni e pericolose evidenziate dalla OWASP Top 10, con potenziali impatti gravi sulla sicurezza e sulla riservatezza dei dati. In questa fase si andranno a testare manualmente tali vulnerabilità sull'applicazione OWASP Juice Shop per verificarne la presenza e dimostrare la possibilità di sfruttamento. Si utilizzeranno metodologie di inserimento di payload malevoli e l'analisi dettagliata delle risposte dell'applicazione e delle sue API per identificare falle di sicurezza rilevanti.

SQL Injection

SQL è il linguaggio usato per comunicare con i database: permette di recuperare, analizzare e modificare i dati. Attraverso le **SQL Injection**, un malintenzionato può inserire comandi SQL nei campi di input di un'applicazione e ottenere l'accesso a informazioni che normalmente non sarebbe autorizzato a visualizzare. Inserendo query malevole nei campi di input, è possibile visualizzare dati riservati o modificare il comportamento dell'applicazione.



Broken Access Control

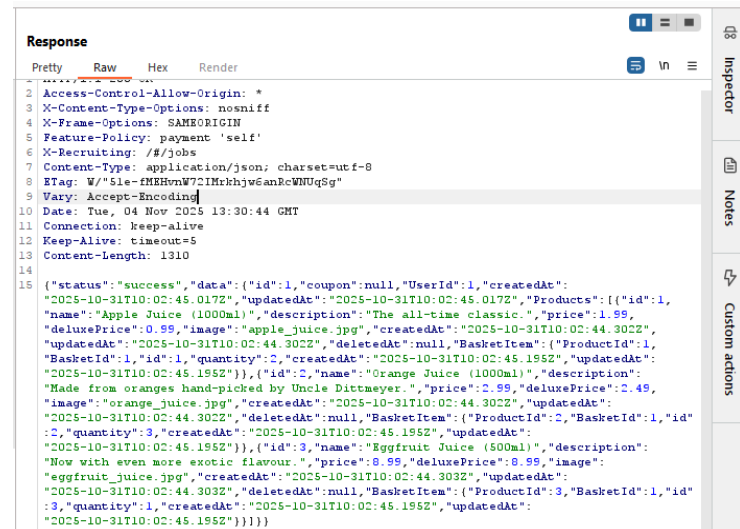
Questa vulnerabilità si classifica secondo [l'OWASP Top 10 come A01:2021-Broken Access Control](#) essendo che rappresenta uno dei rischi di sicurezza più gravi per le applicazioni web. Si manifesta quando non è presente una corretta gestione delle autorizzazioni, permettendo a un attaccante di accedere, modificare o eliminare dati senza avere i permessi necessari.

[illegible]

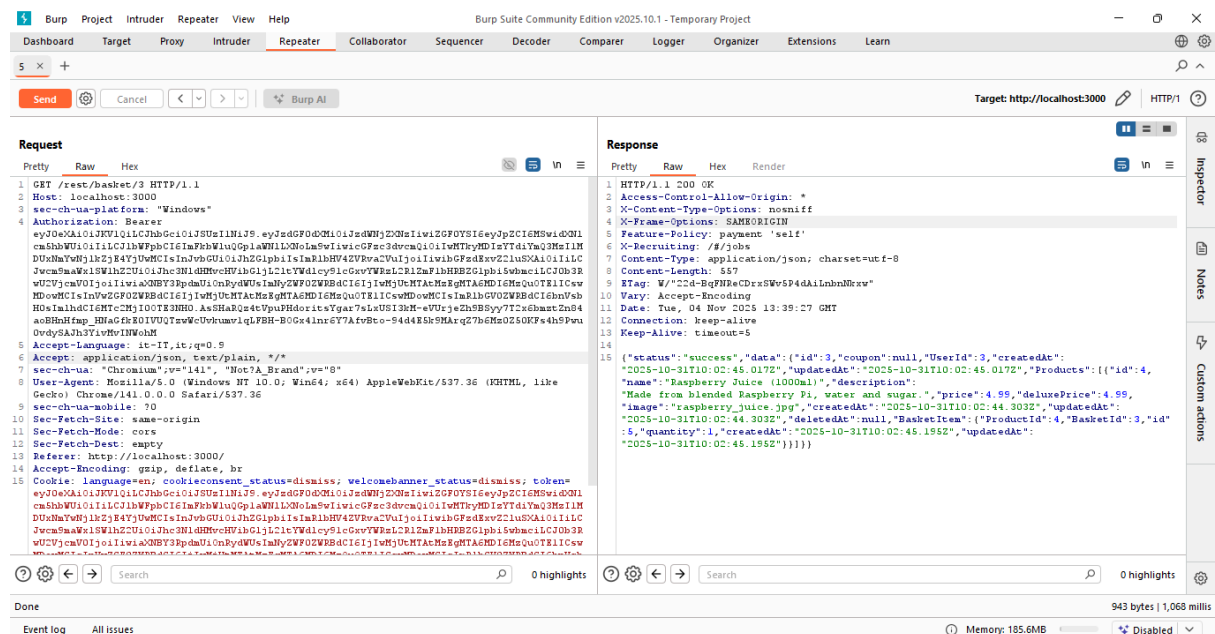
Dopo aver aggiunto prodotti al carrello, con l'ausilio di Burp Suite intercetto la richiesta HTTP e noto che viene utilizzata un'API REST per gestire il carrello. In particolare, vedo che l'endpoint contiene un parametro numerico (in questo caso **1**), che sembra rappresentare l'ID utente o del carrello associato.

A questo punto invio la richiesta al Burp Repeater e osservo che, per l'ID **1**, il carrello contiene tre diversi prodotti con le seguenti quantità:

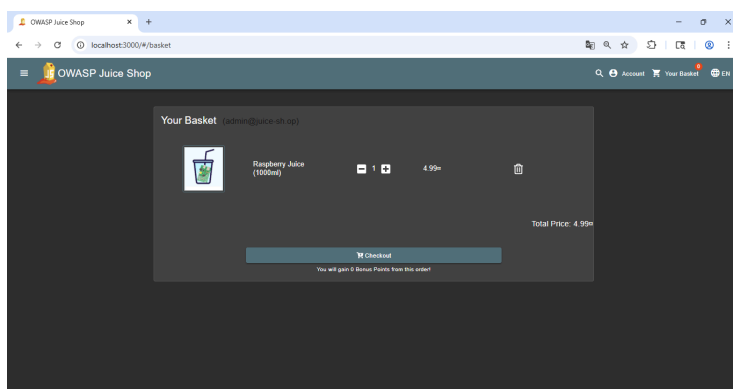
- Apple Juice (1000ml): quantità 2
- Orange Juice (1000ml): quantità 3
- Eggfruit Juice (500ml): quantità 1



Se modifico il parametro ID da **1** a **3** nella richiesta, ottengo comunque una risposta di successo che però, mostra un carrello diverso, contenente un singolo Raspberry Juice (1000ml). Questo suggerisce che sto visualizzando il carrello di un utente diverso da me.



Se ripeto questa operazione in tempo reale, ad esempio tornando alla homepage e osservando nuovamente la sezione “Your Basket” di Juice Shop, intercettando le richieste su Burp e modificando l'ID nel path da **/rest/basket/1** a **/rest/basket/3**, vedrò il contenuto del carrello di un altro utente.



Effettuando il forward di questa richiesta modificata e disattivando l'intercettazione, la sezione “Your Basket” di Juice Shop mostrerà i prodotti appartenenti all'utente con ID 3.

Questo è un chiaro esempio di un Insecure Direct Object Reference (IDOR), un tipo di Broken Access Control, dove non viene effettuata una verifica adeguata sull'ownership delle risorse richieste, permettendo ad un utente di accedere a risorse di altri utenti senza autorizzazione.

Suggerimenti per la correzione di Broken Access Control

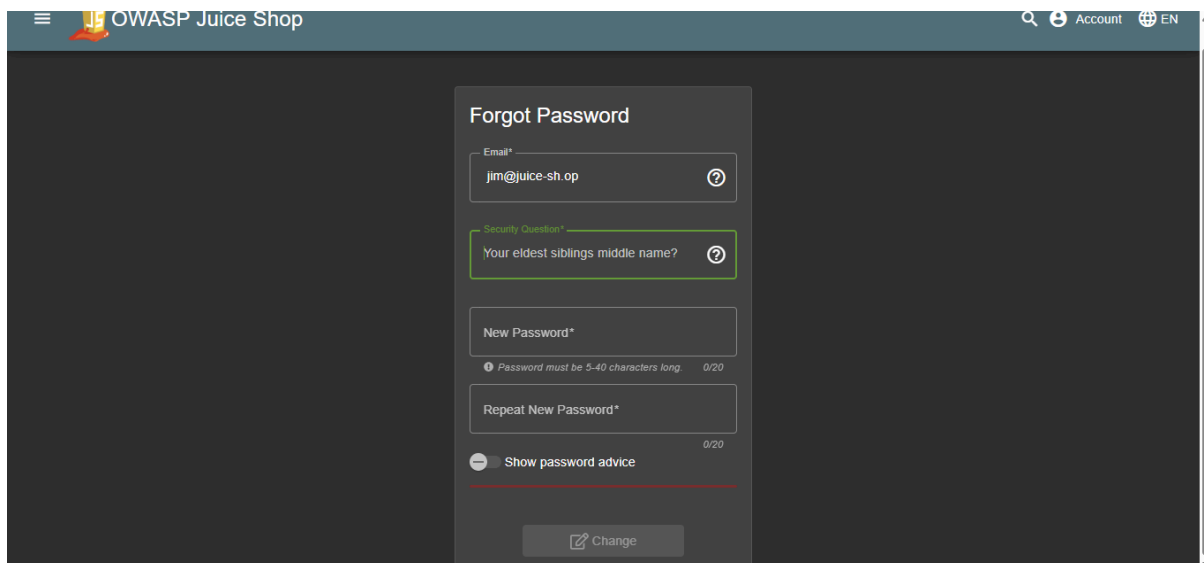
Per correggere la vulnerabilità, si potrebbero implementare controlli di accesso a livello server per verificare rigorosamente i permessi utente per ogni richiesta di risorsa o azione sensibile e non basarsi esclusivamente su controlli lato client.

Un'altra azione potrebbe essere quella di concedere agli utenti solo i permessi strettamente necessari per svolgere le loro funzioni oppure usare l'autenticazione a più fattori per le operazioni sensibili. Implementando queste misure, è possibile mitigare efficacemente la probabilità di sfruttamento di Broken Access Control garantendo che ogni utente possa accedere esclusivamente alle risorse e alle azioni a cui ha diritto.

Sensitive Data Exposure

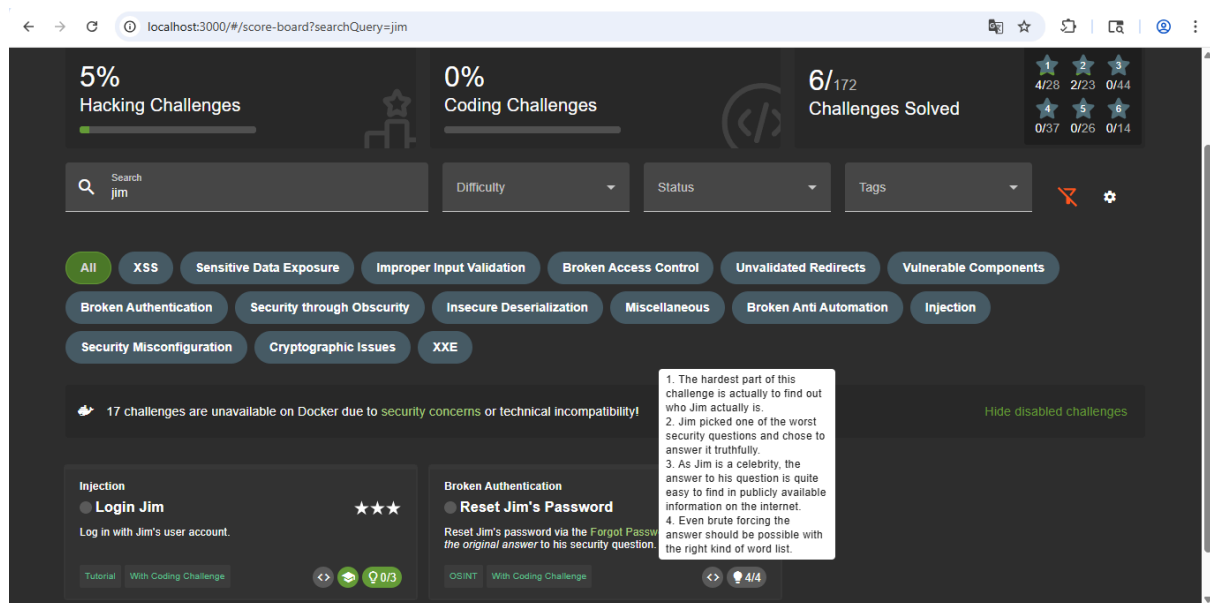
Sensitive Data Exposure (esposizione di dati sensibili), nella terminologia più recente di OWASP [Top 10](#) (2021) questo tipo di rischio è trattato sotto **“Cryptographic Failures”** e occupa la seconda posizione in classifica.

Per testare questa vulnerabilità, mi reco nella sezione di login di juice shop e seleziono **'Forgot your password'**. nel form da compilare per chiedere di resettare la password, vediamo che è presente il campo 'Security Question' che è l'unica forma di autenticazione per la password dimenticata.

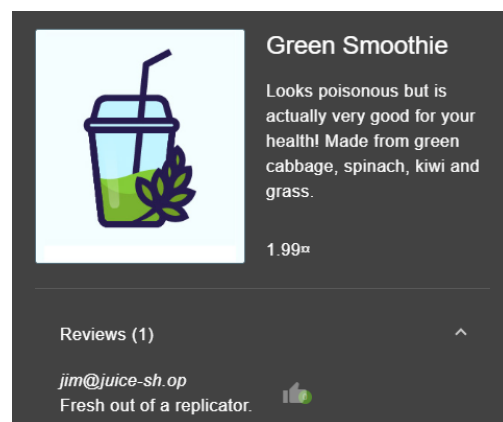
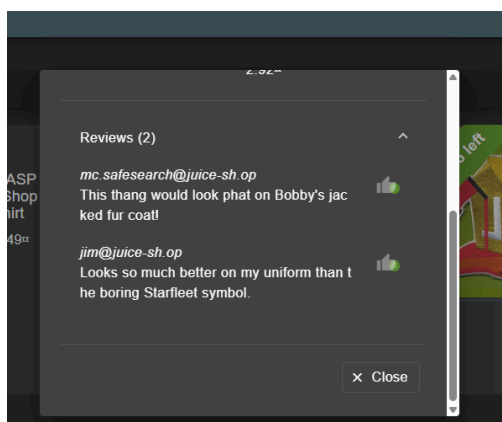


The screenshot shows the 'Forgot Password' form in the OWASP Juice Shop application. The form is titled 'Forgot Password' and contains several input fields. The 'Email*' field is filled with 'jim@juice-sh.op'. The 'Security Question*' field is highlighted with a green border and contains the text 'Your eldest siblings middle name?'. Below this are fields for 'New Password*' and 'Repeat New Password*', both with character counts (0/20). A 'Show password advice' toggle is visible. At the bottom is a 'Change' button.

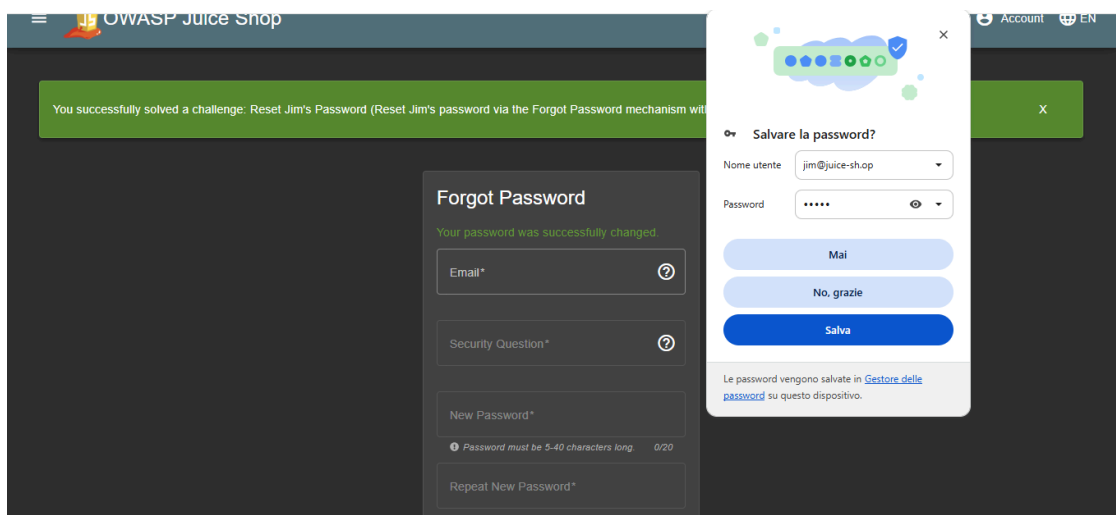
Per trovare il dato richiesto, ovvero il nome del fratello intermedio, navigo sul sito per cercare quante più informazioni possibili. Accedo alla pagina di score-board di juice shop la quale illustra i vari problemi risolvibili sul sito web e trovo delle informazioni riguardo il resettaggio password dell'account di Jim.



Scopro che Jim ha scelto una delle peggiori domande di sicurezza e ha deciso di rispondere sinceramente. Mi viene indicato che Jim è una celebrità e che la risposta alla sua domanda è abbastanza facile da trovare nelle informazioni pubblicamente disponibili su Internet.



Navigando nella pagina che illustra i prodotti presenti su juice shop, vedo delle recensioni lasciate da jim sotto il prodotto OWASP Juice Shop-CTF Velcro Patch e Green Smoothie, i quali mi fanno pensare che questo Jim possa essere l'attore Jim Starfleet e che quindi la risposta alla domanda di sicurezza sia Samuel, ovvero il nome del suo fratello intermedio. A questo punto, compilo il campo della domanda con il nome trovato e inserisco la nuova password che voglio utilizzare ed osservo che l'operazione è andata a buon fine.



Conclusioni finali

In conclusione, il processo di **Vulnerability Assessment e Penetration Test (VAPT)** condotto sull'immagine [Docker di OWASP Juice Shop](#) ha permesso di identificare diverse vulnerabilità critiche che rappresentano potenziali rischi per la sicurezza dell'applicazione.

Sono state adottate tutte le fasi fondamentali del penetration testing, iniziando dalla raccolta di informazioni (Information Gathering), l'analisi delle potenziali vulnerabilità (Vulnerability Assessment), l'esecuzione di tentativi di sfruttamento (Exploitation) e l'analisi delle possibilità di persistenza (Post-Exploitation).

Tra le vulnerabilità emerse, si segnalano in particolare:

- Problemi di Broken Access Control, con evidenze di IDOR che permettono l'accesso non autorizzato a carrelli di altri utenti;
- Configurazioni CORS troppo permissive che espongono l'applicazione a rischi di cross-origin data theft;
- Errori di gestione delle richieste su endpoint REST che generano errori HTTP 500, indice di potenziali vulnerabilità di injection o crash;
- Uso di librerie JavaScript obsolete (es. jQuery 2.2.4) che possono facilitare attacchi XSS.

Tutte le attività sono state accompagnate da una documentazione, comprensiva di output degli strumenti utilizzati, log, screenshot e file raccolti nel pacchetto ZIP allegato.